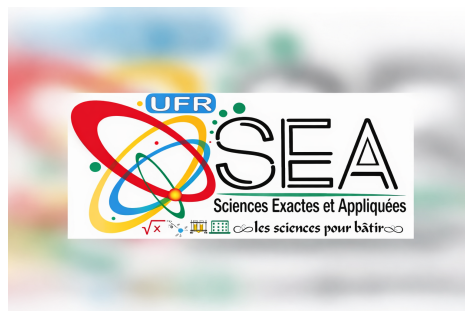




Université Joseph Ki-ZERBO



Unité de formation et de
recherche en
sciences exactes et appliquées
(UFR/SEA)

RAPPORT DE PROJET

Projet IA Symbolique et Prolog

Implémentation et exécution sous
SWISH / SWI-Prolog Notebook

Tronc commun Master 1 Informatique

Membres du groupe :

OUATTARA Barkissa

KABORÉ Paulin

Enseignant :

Dr Wend-Panga Régis Cédric BERE

Outil utilisé : SWISH / SWI-Prolog Notebook

Langage utilisé : Prolog

Lien du notebook : https://swish.swi-prolog.org/p/ia_symbolique.swinb

Année académique : 2025/2026

Table des matières

Introduction générale	3
1 Environnement de travail	3
I Prolog et IA Symbolique	3
2 Exercice 1 : Base de connaissances, faits et règles	4
2.1 Objectif	4
2.2 Code Prolog	4
2.3 Requêtes exécutées	4
2.4 Résultats obtenus	4
2.5 Interprétation	5
3 Exercice 2 : Mini système expert médical	5
3.1 Objectif	5
3.2 Code Prolog	5
3.3 Requêtes exécutées	6
3.4 Résultats obtenus	6
3.5 Interprétation	6
4 Exercice 3 : Ontologie simple en Prolog	6
4.1 Objectif	6
4.2 Code Prolog	7
4.3 Requêtes exécutées	7
4.4 Résultats obtenus	7
4.5 Interprétation	8
5 Exercice 4 : Problème des N reines avec CLP(FD)	8
5.1 Objectif	8
5.2 Code Prolog	8
5.3 Requêtes exécutées	9
5.4 Résultat obtenu	9
5.5 Interprétation	9
II Prolog et IA Neurosymbolique	9
6 Exercice 5 : Raisonnement sur une scène visuelle	10
6.1 Objectif	10
6.2 Code Prolog	10
6.3 Requêtes exécutées	10
6.4 Résultats obtenus	10
6.5 Interprétation	11
III Exercices pratiques	11

7	Exercice 6 : Graphe et chemins	11
7.1	Objectif	11
7.2	Code Prolog	12
7.3	Requêtes exécutées	12
7.4	Résultats obtenus	12
7.5	Interprétation	13
8	Exercice 7 : Coloration de graphe avec CLP(FD)	13
8.1	Objectif	13
8.2	Code Prolog	13
8.3	Requête exécutée	13
8.4	Résultat obtenu	13
8.5	Interprétation	14
9	Exercice 8 : Mini-ILP, apprentissage de la notion de pair	14
9.1	Objectif	14
9.2	Code Prolog	14
9.3	Requêtes exécutées	15
9.4	Résultats obtenus	15
9.5	Interprétation	15
	Conclusion générale	16

Introduction générale

Ce rapport présente un projet pratique réalisé dans le cadre du cours d'Intelligence Artificielle Symbolique et d'introduction à l'IA neurosymbolique. Le travail consiste à utiliser Prolog pour représenter des connaissances sous forme de faits et de règles, puis à interroger ces connaissances par inférence logique.

Le projet est organisé en trois grandes parties :

- **Partie 1 : Prolog et IA Symbolique ;**
- **Partie 2 : Prolog et IA Neurosymbolique ;**
- **Partie 3 : Exercices pratiques.**

Le travail a été exécuté avec **SWISH**, l'environnement web de SWI-Prolog. Les captures d'écran des exécutions sont insérées dans le rapport ou laissées en espace réservé lorsqu'elles doivent encore être ajoutées.

1 Environnement de travail

- Langage : Prolog ;
- Outil : SWISH / SWI-Prolog Notebook ;
- Bibliothèque utilisée : `library(clpfd)` pour les contraintes sur domaines finis ;
- Navigateur : à compléter ;
- Lien du notebook SWISH : [A_COMPLETER_AVEC_LE_LIEN_SWISH](#).

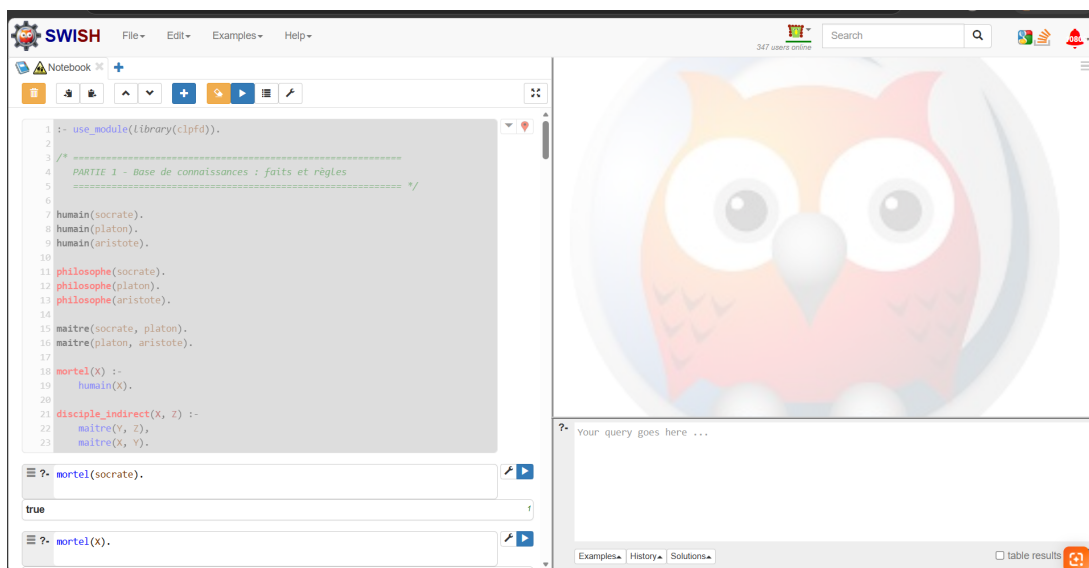


FIGURE 1 – Résultat des requêtes mortel et disciple_indirect dans SWISH

Première partie

Prolog et IA Symbolique

Cette première partie regroupe les exercices qui montrent le rôle de Prolog comme outil d'IA symbolique. Les connaissances sont écrites explicitement sous forme de faits et

de règles. Les réponses sont obtenues par raisonnement logique.

2 Exercice 1 : Base de connaissances, faits et règles

2.1 Objectif

L'objectif est de représenter une base de connaissances simple contenant des humains, des philosophes et une relation de maître. La règle `mortel(X)` permet de déduire qu'un individu est mortel lorsqu'il est humain.

2.2 Code Prolog

```
1 humain(socrate).
2 humain(platon).
3 humain(aristote).
4
5 philosophe(socrate).
6 philosophe(platon).
7 philosophe(aristote).
8
9 maitre(socrate, platon).
10 maitre(platon, aristote).
11
12 mortel(X) :-
13     humain(X).
14
15 disciple_indirect(X, Z) :-
16     maitre(Y, Z),
17     maitre(X, Y).
```

2.3 Requêtes exécutées

```
1 mortel(socrate).
2 mortel(X).
3 disciple_indirect(X, aristote).
```

2.4 Résultats obtenus

- `mortel(socrate).` retourne `true`;
- `mortel(X).` retourne `X = socrate, X = platon, X = aristote`;
- `disciple_indirect(X, aristote).` retourne `X = socrate`.



FIGURE 2 – Résultat des requêtes mortel et disciple_indirect dans SWISH

2.5 Interprétation

Prolog utilise la règle `mortel(X) :- humain(X).` pour conclure que tous les individus déclarés comme humains sont mortels. La règle `disciple_indirect` permet aussi de déduire que Socrate est disciple indirect d'Aristote, car Socrate a pour maître Platon et Platon a pour maître Aristote.

3 Exercice 2 : Mini système expert médical

3.1 Objectif

L'objectif est de construire un mini système expert. Les symptômes sont représentés par des faits, et les diagnostics sont obtenus par des règles de type « si symptômes alors maladie ».

3.2 Code Prolog

```

1 symptome(patient1, fièvre).
2 symptome(patient1, toux).
3 symptome(patient1, courbatures).
4
5 symptome(patient2, fièvre).
6 symptome(patient2, eruption_cutanee).
7
8 diagnostic(Patient, grippe) :-
9     symptome(Patient, fièvre),
10    symptome(Patient, toux),
11    symptome(Patient, courbatures).
12
13 diagnostic(Patient, rougeole) :-
14     symptome(Patient, fièvre),
15     symptome(Patient, eruption_cutanee).
16

```

```

17 diagnostic(Patient, inconnu) :-
18     symptome(Patient, _),
19     \+ diagnostic(Patient, grippe),
20     \+ diagnostic(Patient, rougeole).

```

3.3 Requêtes exécutées

```

1 diagnostic(patient1, D).
2 diagnostic(patient2, D).
3 findall(P-D, diagnostic(P, D), Liste).

```

3.4 Résultats obtenus

- `diagnostic(patient1, D).` donne `D = grippe`;
- `diagnostic(patient2, D).` donne `D = rougeole`;
- `findall(P-D, diagnostic(P,D), Liste).` retourne la liste des diagnostics trouvés.

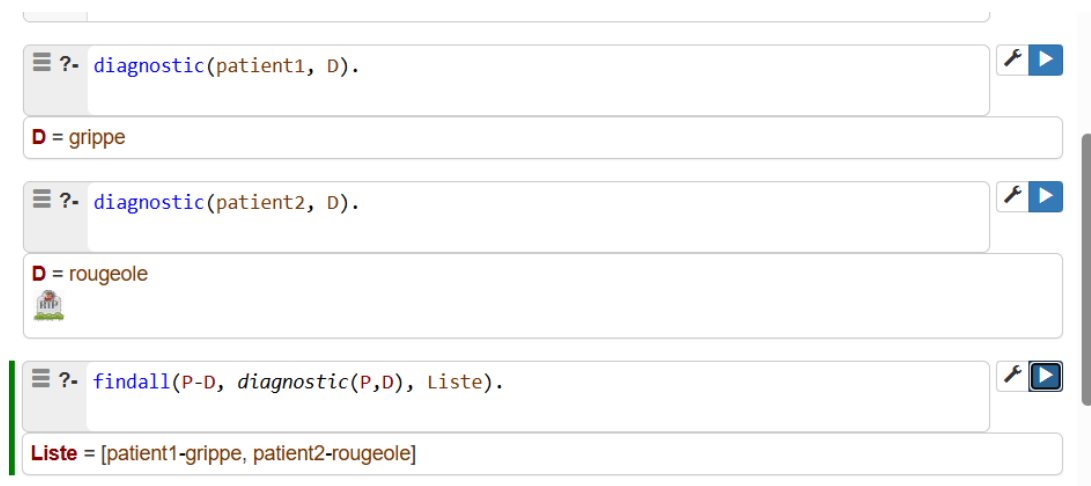


FIGURE 3 – Résultat des requêtes du système expert médical

3.5 Interprétation

Le patient 1 présente la fièvre, la toux et les courbatures : le système déduit donc la grippe. Le patient 2 présente la fièvre et une éruption cutanée : le système déduit donc la rougeole. Ce type de raisonnement correspond au fonctionnement classique des systèmes experts.

4 Exercice 3 : Ontologie simple en Prolog

4.1 Objectif

L'objectif est de représenter une hiérarchie de concepts. Le prédicat `sous_classe` décrit les relations entre classes et `instance_de` relie un individu à une classe.

4.2 Code Prolog

```
1 sous_classe(animal, etre_vivant).
2 sous_classe(mammifere, animal).
3 sous_classe(primare, mammifere).
4 sous_classe(humain, primare).
5 sous_classe(chien, mammifere).
6
7 instance_de(socrate, humain).
8 instance_de(rex, chien).
9
10 est_un(X, Y) :-
11     instance_de(X, Y).
12
13 est_un(X, Y) :-
14     instance_de(X, Z),
15     est_un(Z, Y).
16
17 est_un(X, Y) :-
18     sous_classe(X, Y).
19
20 est_un(X, Y) :-
21     sous_classe(X, Z),
22     est_un(Z, Y).
```

4.3 Requêtes exécutées

```
1 est_un(socrate, animal).
2 est_un(rex, primare).
3 findall(C, est_un(socrate, C), Classes).
```

4.4 Résultats obtenus

- `est_un(socrate, animal).` retourne `true`;
- `est_un(rex, primare).` retourne `false`;
- `findall(C, est_un(socrate, C), Classes).` retourne les classes auxquelles appartient Socrate.

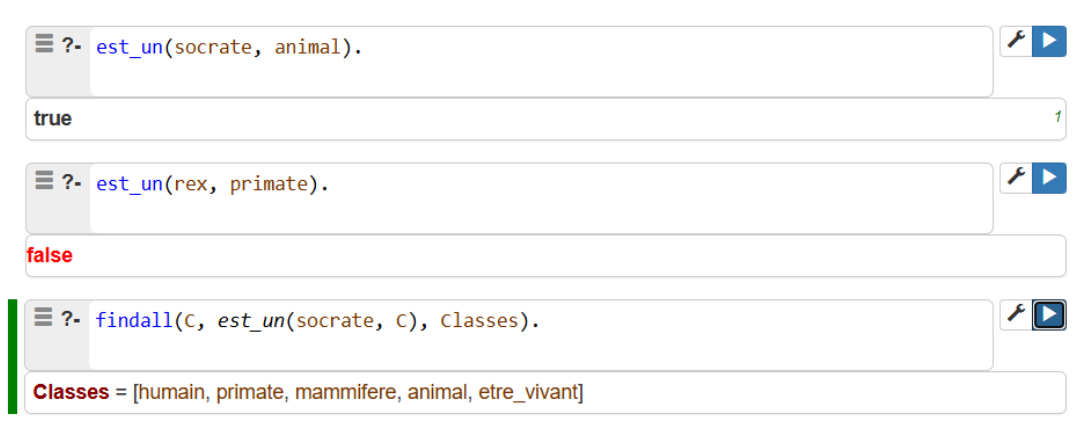


FIGURE 4 – Résultat des requêtes sur l'ontologie simple

4.5 Interprétation

Socrate est une instance de `humain`. Comme `humain` est une sous-classe de `primate`, puis de `mammifere`, puis de `animal`, Prolog peut déduire automatiquement que Socrate appartient aussi à ces classes générales.

5 Exercice 4 : Problème des N reines avec CLP(FD)

5.1 Objectif

L'objectif est d'utiliser la programmation logique par contraintes pour placer N reines sur un échiquier de taille N, sans qu'elles soient sur la même ligne ni sur la même diagonale.

5.2 Code Prolog

```

1 :- use_module(library(clpfd)).
2
3 n_reines(N, Reines) :-
4     length(Reines, N),
5     Reines ins 1..N,
6     all_different(Reines),
7     no_attaque(Reines),
8     labeling([], Reines).
9
10 no_attaque([]).
11
12 no_attaque([R|Rs]) :-
13     no_attaque(R, Rs, 1),
14     no_attaque(Rs).
15
16 no_attaque(_, [], _).
17
18 no_attaque(R, [R1|Rs], D) :-
19     R1 #\= R + D,
20     R1 #\= R - D,

```

```

21     D1 is D + 1,
22     no_attaque(R, Rs, D1).

```

5.3 Requêtes exécutées

```

1 n_reines(4, R).
2 n_reines(8, Qs).

```

5.4 Résultat obtenu

Pour `n_reines(4, R)`., une solution possible est :

```

1 R = [2, 4, 1, 3].

```



FIGURE 5 – Résultat de la résolution du problème des N reines

5.5 Interprétation

Chaque position de la liste représente la ligne occupée par une reine dans une colonne. Les contraintes `all_different` empêchent deux reines d'être sur la même ligne, et les contraintes `#` empêchent les attaques diagonales.

Deuxième partie

Prolog et IA Neurosymbolique

Cette deuxième partie montre comment des données symboliques peuvent représenter le résultat d'un module de perception. Dans une architecture neurosymbolique réelle, un réseau de neurones pourrait reconnaître les objets et leurs propriétés, tandis que Prolog réaliserait le raisonnement logique.

6 Exercice 5 : Raisonnement sur une scène visuelle

6.1 Objectif

L'objectif est de représenter une scène simple contenant des objets, des couleurs, des formes, des tailles et des relations spatiales. Des règles permettent ensuite de répondre à des questions symboliques.

6.2 Code Prolog

```
1 objet(o1).
2 objet(o2).
3 objet(o3).
4
5 couleur(o1, rouge).
6 couleur(o2, bleu).
7 couleur(o3, rouge).
8
9 forme(o1, cube).
10 forme(o2, sphere).
11 forme(o3, sphere).
12
13 taille(o1, grand).
14 taille(o2, petit).
15 taille(o3, grand).
16
17 a_gauche(o1, o2).
18 a_gauche(o2, o3).
19
20 rouge_et_grand(X) :-
21     couleur(X, rouge),
22     taille(X, grand).
23
24 cube_gauche_sphere_bleue :-
25     forme(C, cube),
26     forme(S, sphere),
27     couleur(S, bleu),
28     a_gauche(C, S).
```

6.3 Requêtes exécutées

```
1 rouge_et_grand(X).
2 findall(X, rouge_et_grand(X), L), length(L, N).
3 cube_gauche_sphere_bleue.
```

6.4 Résultats obtenus

— rouge_et_grand(X). retourne X = o1 puis X = o3;

- `findall(X, rouge_et_grand(X), L), length(L, N).` retourne `L = [o1,o3]` et `N = 2`;
- `cube_gauche_sphere_bleue.` retourne `true`.

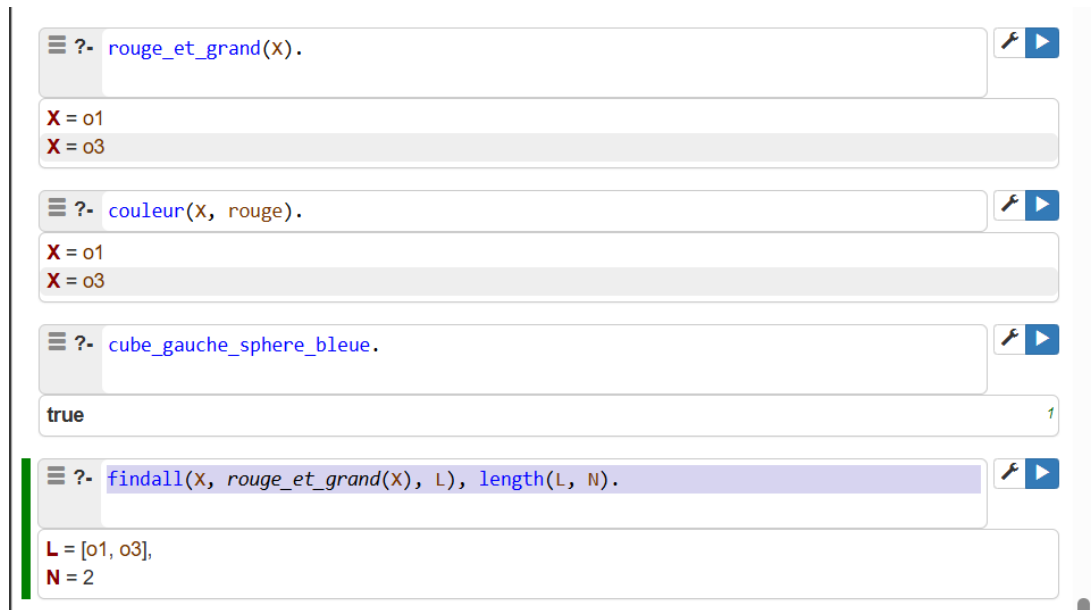


FIGURE 6 – Résultat des requêtes sur la scène visuelle

6.5 Interprétation

Les objets `o1` et `o3` sont rouges et grands. La règle `cube_gauche_sphere_bleue` est vraie, car il existe un cube à gauche d'une sphère bleue. Cette partie illustre le principe neurosymbolique : la perception fournit les faits, et le raisonnement symbolique produit les réponses.

Troisième partie

Exercices pratiques

Cette troisième partie regroupe les exercices pratiques du tutoriel : recherche de chemins dans un graphe, coloration de graphe avec contraintes et simulation d'une règle apprise par apprentissage logique inductif.

7 Exercice 6 : Graphe et chemins

7.1 Objectif

L'objectif est de modéliser un graphe orienté et de trouver les chemins possibles entre deux sommets.

7.2 Code Prolog

```
1 arete(a, b).
2 arete(b, c).
3 arete(c, d).
4 arete(a, c).
5 arete(b, d).
6
7 chemin(X, Y, [X, Y]) :-
8     arete(X, Y).
9
10 chemin(X, Y, [X|Chemin]) :-
11     arete(X, Z),
12     Z \= Y,
13     chemin(Z, Y, Chemin).
```

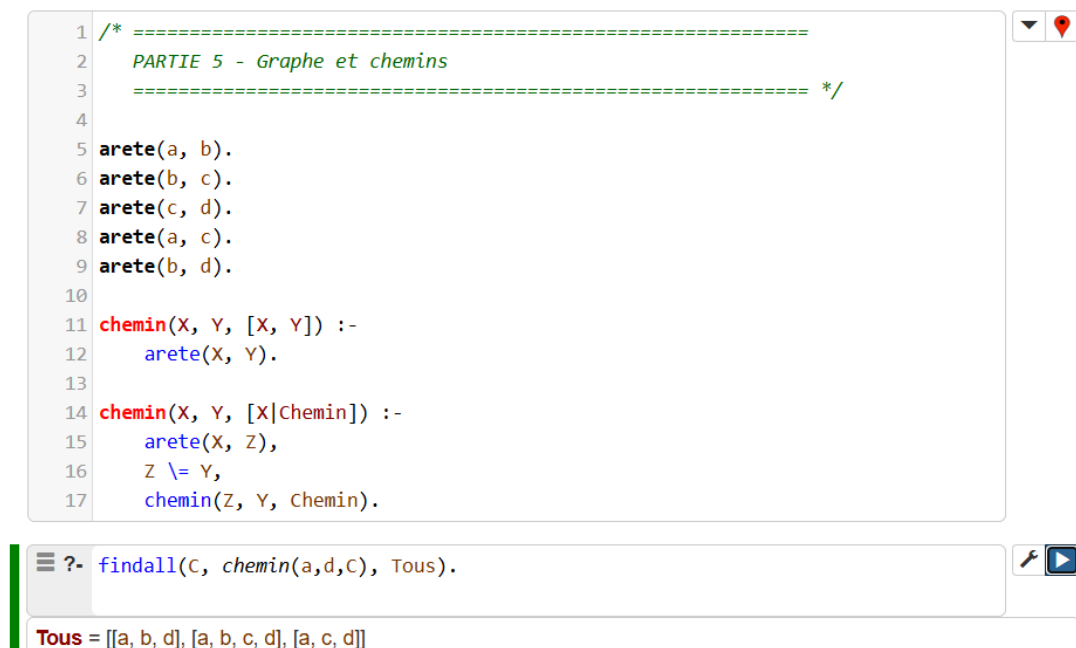
7.3 Requêtes exécutées

```
1 chemin(a, d, C).
2 findall(C, chemin(a, d, C), Tous).
```

7.4 Résultats obtenus

Les chemins possibles de a vers d sont :

```
1 [a, b, d]
2 [a, b, c, d]
3 [a, c, d]
```



The screenshot shows a Prolog IDE with a code editor and a console. The code editor contains the Prolog code from section 7.2. The console shows the execution of the query `findall(C, chemin(a,d,C), Tous).` and the resulting list `Tous = [[a, b, d], [a, b, c, d], [a, c, d]]`.

```
1 /* =====
2  PARTIE 5 - Graphe et chemins
3  ===== */
4
5 arete(a, b).
6 arete(b, c).
7 arete(c, d).
8 arete(a, c).
9 arete(b, d).
10
11 chemin(X, Y, [X, Y]) :-
12     arete(X, Y).
13
14 chemin(X, Y, [X|Chemin]) :-
15     arete(X, Z),
16     Z \= Y,
17     chemin(Z, Y, Chemin).
```

`?- findall(C, chemin(a,d,C), Tous).`

`Tous = [[a, b, d], [a, b, c, d], [a, c, d]]`

FIGURE 7 – Résultat des chemins de a vers d

7.5 Interprétation

Prolog explore les arêtes disponibles et construit récursivement les chemins possibles. La requête `findall` permet de regrouper toutes les solutions dans une seule liste.

8 Exercice 7 : Coloration de graphe avec CLP(FD)

8.1 Objectif

L'objectif est de colorier une carte simplifiée avec trois couleurs, en imposant que deux pays voisins n'aient pas la même couleur.

8.2 Code Prolog

```
1 :- use_module(library(clpfd)).
2
3 colorier(Carte) :-
4     Carte = [France, Belgique, Allemagne, Espagne, Italie, PaysBas
5             ],
6     Carte ins 1..3,
7     France #\= Belgique,
8     France #\= Allemagne,
9     France #\= Espagne,
10    France #\= Italie,
11    Belgique #\= PaysBas,
12    Belgique #\= Allemagne,
13    labeling([], Carte).
```

8.3 Requête exécutée

```
1 colorier([F, B, A, E, I, P]).
```

8.4 Résultat obtenu

SWISH retourne plusieurs solutions possibles. La première solution obtenue peut être :

```
1 A = 3,
2 B = E, E = I, I = 2,
3 F = P, P = 1.
```

Cela correspond à :

```
1 France      = 1
2 Belgique    = 2
3 Allemagne   = 3
4 Espagne     = 2
5 Italie      = 2
6 PaysBas     = 1
```

Donc :

```
1 Carte = [1, 2, 3, 2, 2, 1].
```

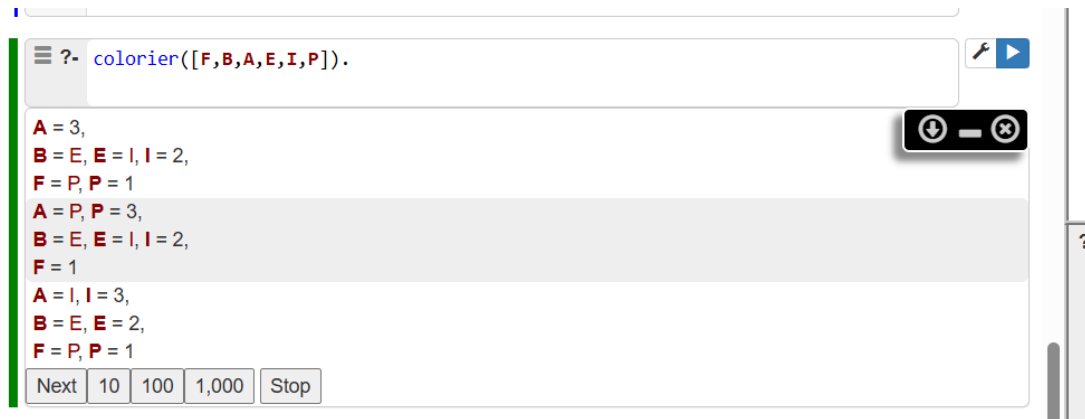


FIGURE 8 – Résultat de la coloration de graphe dans SWISH

8.5 Interprétation

Chaque nombre représente une couleur. Les contraintes garantissent que deux pays voisins reçoivent des couleurs différentes. Prolog peut retourner plusieurs solutions, car plusieurs colorations respectent les contraintes.

9 Exercice 8 : Mini-ILP, apprentissage de la notion de pair

9.1 Objectif

L'objectif est de simuler le principe de l'apprentissage logique inductif. On écrit manuellement la règle qu'un système ILP pourrait apprendre pour reconnaître les nombres pairs.

9.2 Code Prolog

```
1 zero(0).
2
3 suivant(0, 1).
4 suivant(1, 2).
5 suivant(2, 3).
6 suivant(3, 4).
7 suivant(4, 5).
8 suivant(5, 6).
9 suivant(6, 7).
10 suivant(7, 8).
11 suivant(8, 9).
12 suivant(9, 10).
13
14 pair(0).
```

```

15 pair(X) :-
16     suivant(X1, X),
17     suivant(X2, X1),
18     pair(X2).

```

9.3 Requêtes exécutées

```

1 pair(4).
2 pair(5).
3 pair(X).

```

9.4 Résultats obtenus

- `pair(4)`. retourne `true`;
- `pair(5)`. retourne `false`;
- `pair(X)`. retourne les nombres pairs définis dans la base.

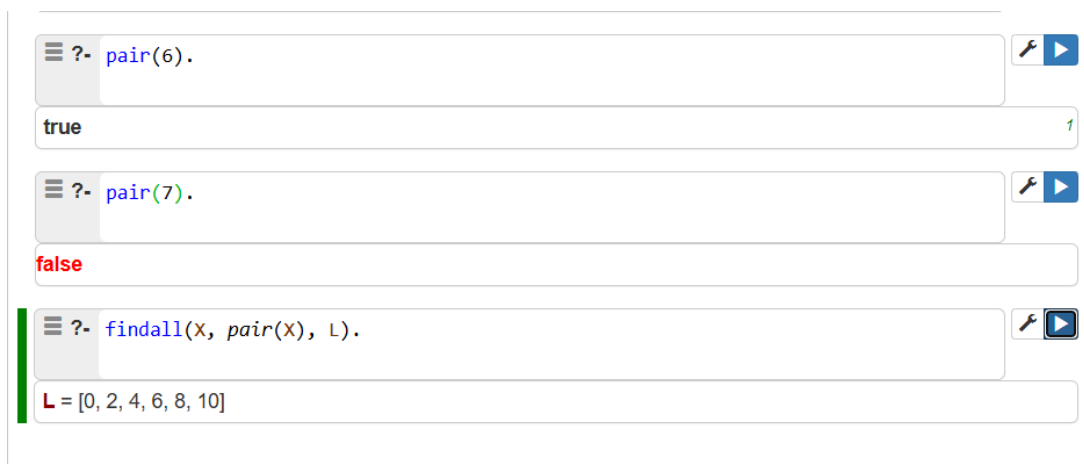


FIGURE 9 – Résultat des requêtes sur le prédicat `pair`

9.5 Interprétation

La règle indique qu'un nombre est pair si le nombre situé deux positions avant lui est pair. Comme `pair(0)` est donné comme fait de base, Prolog peut déduire progressivement `pair(2)`, `pair(4)`, `pair(6)`, etc.

Conclusion générale

Ce projet a permis de mettre en pratique plusieurs aspects de l'IA symbolique et neurosymbolique. Dans la première partie, Prolog a été utilisé pour représenter des faits, des règles, un système expert, une ontologie et un problème de contraintes. Dans la deuxième partie, le raisonnement sur des scènes visuelles a illustré l'idée neurosymbolique : combiner perception et logique. Dans la troisième partie, les exercices pratiques ont montré l'utilisation de Prolog pour les graphes, les contraintes et la définition récursive d'une notion logique.

L'utilisation de SWISH facilite l'exécution directe des programmes et la production de résultats exploitables dans un rapport. Les captures d'écran ajoutées au document permettent de vérifier que les requêtes ont bien été exécutées.